PARTE 2. PROGRAMACIÓN FUNCIONAL.

Ejercicio 2 (1 punto)

Sin definir otras funciones adicionales, implementar una función en lisp llamada contar_ceros que, dada una lista de listas no anidadas, devuelva una nueva lista con el número de ceros en cada una de las listas originales.

(contar_ceros '((1 2 3) (0 2 3) (1 2 3))) -> (0 1 0)

(contar_ceros '(NIL (0 0 0) (1 2 0))) -> (0 3 1)

Ejercicio 3 (2 puntos)

Sin definir otras funciones adicionales, implementar una función llamada aplicar_funcion que tome como entrada una lista que pudiera ser anidada y aplique la función a cada uno de los átomos que aparezcan en cualquier nivel de anidamiento.

(aplicar_funcion #'1+ '(1 2 3 (1 2))) -> (2 3 4 (2 3))

(aplicar_funcion #'1+ '((1 2) (1 2 (1 2) 3) 3)) -> ((2 3) (2 3 (2 3) 4) 4)

(aplicar_funcion (lambda(x) (* 2 x)) '((1 2) (1 2 (1 2) 3) 3)) -> ((2 4) (2 4 (2 4) 6) 6)

Ejercicio 4 (2 puntos)

Sin definir otras funciones adicionales, implementar una función en lisp llamada aplicar_binaria que tome como entrada una función binaria y un número arbitrario de valores numéricos y devuelva como resultado una lista con los resultados de aplicar la función sobre los argumentos dos a dos, devolviendo NIL en la última posición cuando el número de elementos sea impar.

(aplicar_binaria #'+ 1 2 3 4 5) -> (3 7 NIL)

(aplicar_binaria #'* 1 2 3 4) -> (2 12)